NexLink

AI-Assisted Private Networking, Network Diagnostics and Remote Administration Platform

Project Status Report

What is built, what is verified, and what remains

Item	Detail
Prepared for	Maitrey Bharambe
Date	18 October 2026
Repository	github.com/Maitrey-Bharambe/nexlinkk
Stage	Foundation complete; real WireGuard tunnel verified in a single-host lab
Basis of this report	Results of test runs and live checks performed on 16–18 October 2026. Nothing is reported as done unless it was tested.

Contents

1. Executive summary 3
Key numbers 3
2. Goals and scope 4
Technology stack 4
3. Architecture 5
Components 5
Five layers of “is it connected?” 5
Enrollment flow 6
Peer link states 6
Diagnostic loop 7
Data model 8
4. What is in the project 9
4.1 Web console 9
4.2 Control plane 9
4.3 Networking agent (Go) 9
4.4 Diagnostic service (Python) 9
5. Status matrix 11
6. Verification results 12
6.1 Automated tests 12
6.2 Real WireGuard tunnel (lab) 12
6.3 Browser checks 14
6.4 Bugs found and fixed during verification 14
7. Security model 15
8. What is not in the project yet 16
Known limitations and environment notes 16
9. How to run 17
Roadmap 17
Appendix A — Repository layout 18
Appendix B — Documentation index 18

1. Executive summary

NexLink is a working foundation for a private-networking platform: a web console, a control plane on Node.js and MongoDB, an endpoint agent written in Go, and an optional Python diagnostic service that can use a Groq-hosted LLM. Devices enroll with their own cryptographic identity, report signed telemetry, and are shown as connected only when WireGuard reports a fresh handshake.

Since the last update the remaining networking step has been advanced: a real WireGuard tunnel between two enrolled agents now runs end to end on one machine (nexlink-agent lab). The console showed the link as connected and direct, and an approved connectivity test executed by the agent through the encrypted tunnel was verified at 0.23 ms RTT with 0% packet loss. The test that still needs a second physical PC is documented step by step.

Key numbers

Measure	Result	Notes
Automated tests passing	94 / 94	Control plane 59, Go agent 20, diagnostic service 15
Go agent ↔ live control plane integration test	Passed	Enroll, token replay rejected, telemetry, heartbeat loop
Real WireGuard tunnel (lab, two agents)	Verified	75 s and 8 min runs, zero errors; 94 RTT samples, typically under 1 ms
Approved connectivity test through the tunnel	Successful	RTT 0.23 ms, jitter 0.31 ms, loss 0%, 3/3 measurements valid
Console production build	Passed	12 routes, Next.js 15
Bugs found and fixed during verification	11	See section 6.4

Where the project stands

Done and tested: accounts, roles, networks, enrollment, signed agent protocol, telemetry and honest health, topology, incidents, policy-gated actions, audit, onboarding, Go agent, real WireGuard tunnel (lab).

Ready, manual run pending: tunnel between two physical PCs; kernel / WireGuardNT interface via the WireGuard app; live Groq diagnosis (needs your API key).

Not started: relay fallback and NAT traversal, TLS deployment guide, MFA, OS keychain key storage, CI pipeline.

2. Goals and scope

The brief asked for a serious Computer Networks and AI-systems project rather than a dashboard. The design follows these rules, and the tests check them:

- **Control plane and data plane are separate.** The web server coordinates keys, addresses and policy; VPN packets travel directly between devices over WireGuard.
- **No fabricated state.** Missing data is shown as unknown or not measured, never as zero or healthy; simulated data can never make a device healthy.
- **Identity is cryptographic.** Devices prove possession of their own Ed25519 key; private keys never leave the device.
- **AI is advisory and gated.** Diagnoses must cite collected evidence; actions come from an allowlist, pass a deterministic policy engine, and need human approval.
- **Host changes are opt-in.** The agent is read-only unless privileged mode is enabled locally on that machine.

Technology stack

Layer	Technology	Role
Web console	Next.js 15 (App Router, JavaScript), React 19, Tailwind v4, React Flow, Socket.IO client, lucide-react	Operator UI: overview, networks, devices, topology, diagnostics, approvals, audit, onboarding
Control plane	Node.js 22, Express 4, Zod, Mongoose 8, Socket.IO, pino	Authoritative config, auth, RBAC, enrollment, telemetry, incidents, policy, audit
Database	MongoDB (local, Docker or Atlas)	State with unique / partial / TTL indexes as integrity constraints
Networking agent	Go 1.27, wgctrl, wireguard-go + gVisor netstack	Identity, signed transport, WireGuard inspect/apply, probes, executor, lab
VPN	WireGuard	Encrypted data plane (no custom cryptography)
Diagnostic service	Python 3.13, FastAPI, Pydantic, httpx; Groq API	Structured diagnosis contracts; mock and Groq providers

3. Architecture

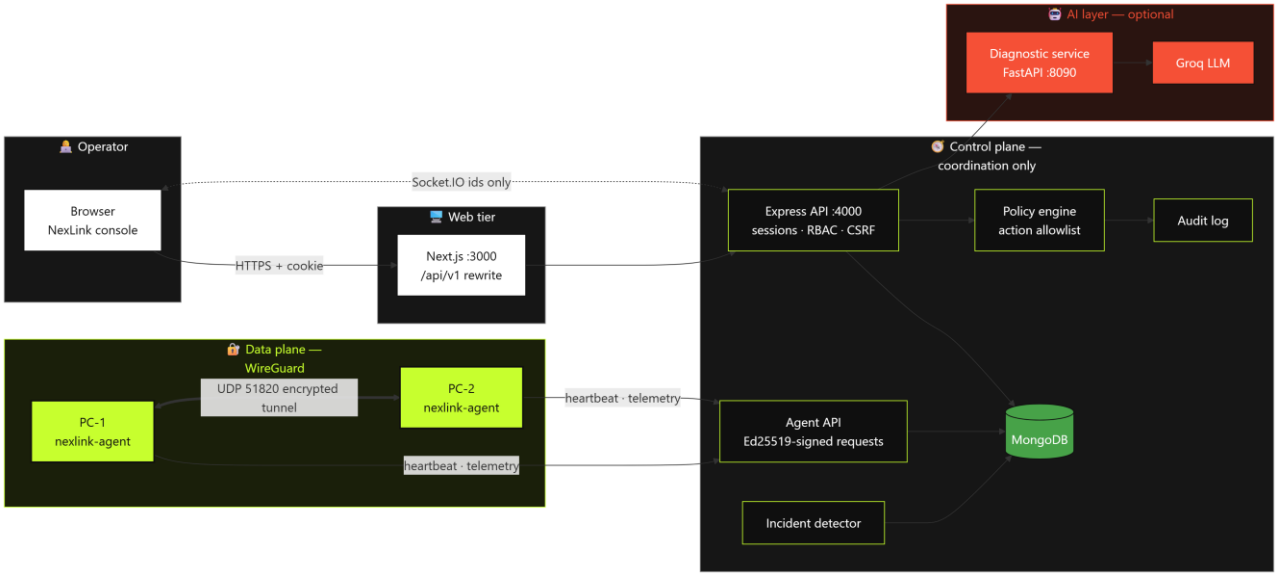


Figure 1 — Components. The thick lime link is the only path VPN traffic takes.

Components

Component	Location	Responsibility
Console	frontend/	Same-origin REST via /api/v1 rewrite; live updates via Socket.IO (ids only, data refetched through authorised REST)
User API	control-plane/src/modules/*	Sessions (HttpOnly cookie), CSRF, RBAC, scoped access, validation with strict schemas
Agent API	control-plane/src/modules/agent	Separate auth path: Ed25519-signed requests with timestamp window and nonce replay protection
Incident detector	modules/incidents/detector.js	Every 30 s evaluates heartbeats, interface state, handshakes, loss and latency
Policy engine	modules/policies	Allowlist, parameter schemas, device state, capability, rate limits, approver permission
Networking agent	networking-agent/	Enroll, heartbeat, telemetry, probes, approved commands, WireGuard inspect/apply, doctor, export-config, lab
Diagnostic service	diagnostic-service/	Provider interface (mock, Groq); grounding validator; called only by the control plane

Five layers of “is it connected?”

None of these is inferred from the layer above it:

Layer	Meaning	Evidence used
1 Registered	Identity key enrolled	devices.enrollmentState = enrolled
2 Configured	Desired peers computed	networks.configVersion and GET /agent/config
3 Tunnel up	WireGuard completed a handshake	Last handshake ≤ 180 s at report time
4 Reachable	Packets cross the tunnel	TCP probe to the peer’s tunnel address
5 Application works	A service on the peer answers	Not measured yet

Enrollment flow

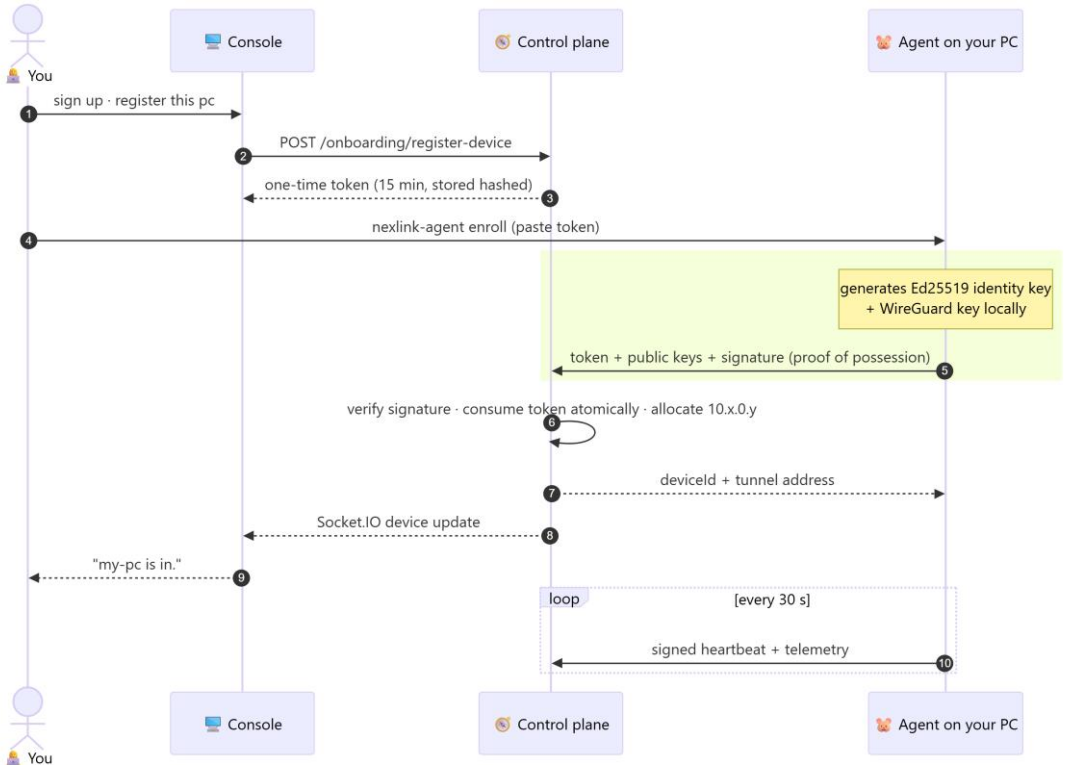


Figure 2 — Enrolling a PC: one-time token, locally generated keys, proof of possession.

Peer link states



Figure 3 — States the topology draws; “configured” is never shown as a working tunnel.

Diagnostic loop

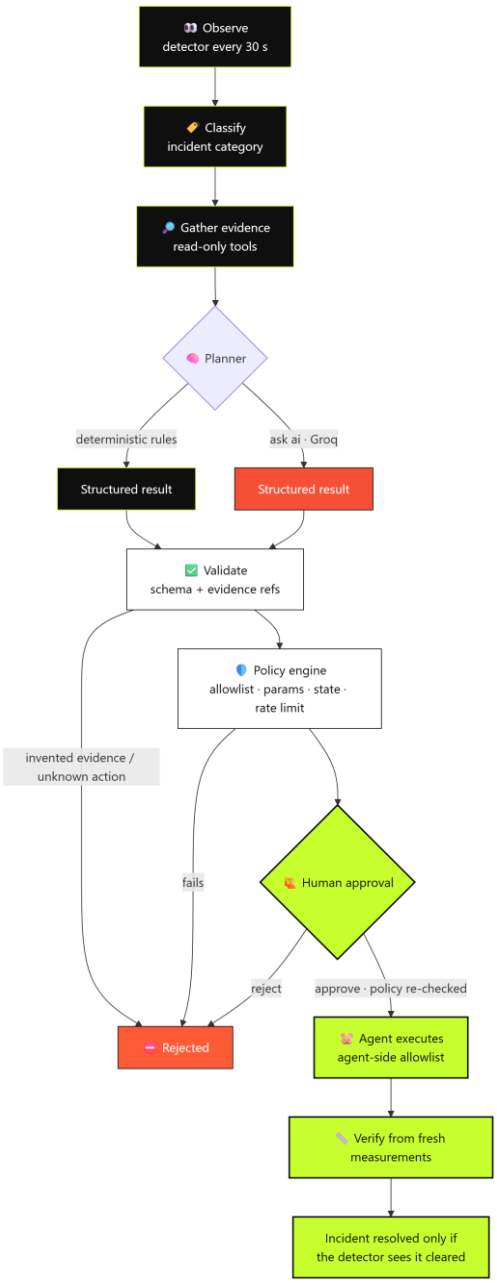


Figure 4 — Observe → evidence → plan → validate → policy → human approval → execute → verify.

Data model

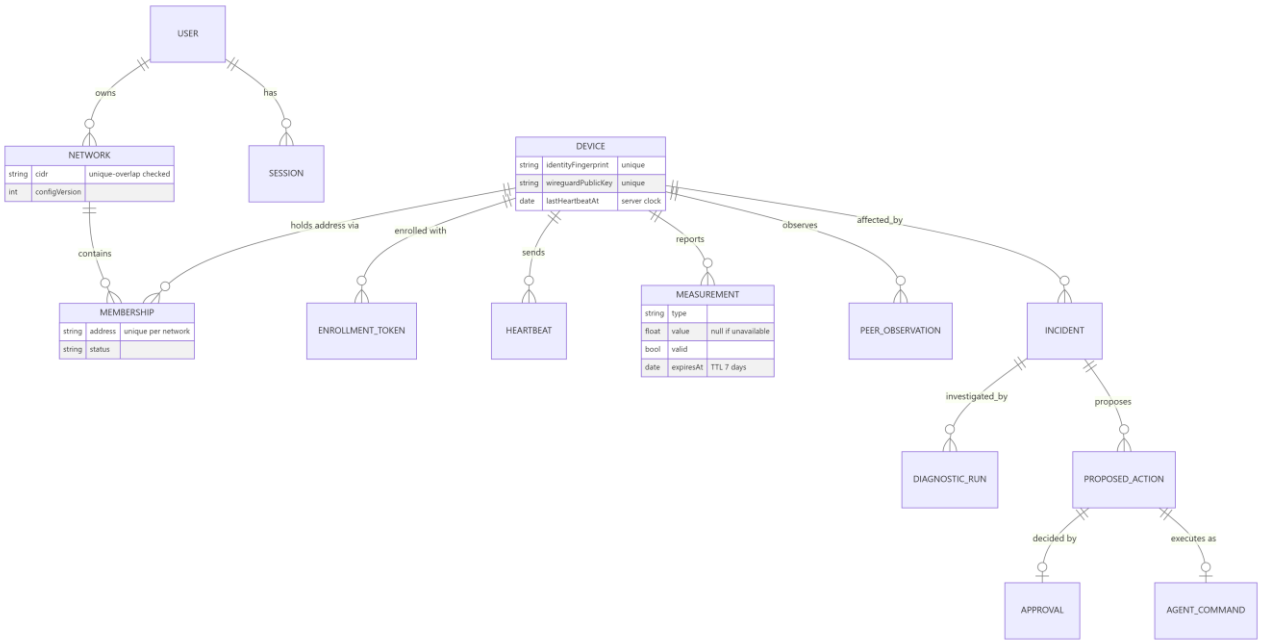


Figure 5 — Main MongoDB collections and relationships.

4. What is in the project

4.1 Web console

- **Sign in / sign up** — first account becomes admin, later accounts are operators who see only their own networks.
- **Register this PC** — creates the user's private network (free 10.x.0.0/24) if needed, issues a one-time token, and flips to "registered" when the agent enrolls.
- **Overview** — counts computed from records; honest empty states.
- **Networks** — create with CIDR validation, inspect, share, request deletion.
- **Devices** — three independent signals per device: agent ↔ control plane, WireGuard interface, peer tunnels; history; revoke; remove membership.
- **Topology** — React Flow graph; solid lime only for links with a fresh handshake.
- **Diagnostics** — incidents, evidence, deterministic or AI diagnosis, proposed actions with policy verdicts.
- **Approvals and audit log** — approve/reject queue; searchable, append-only audit trail.

4.2 Control plane

- Authentication: scrypt password hashing, HttpOnly SameSite=Strict session cookie, CSRF token, lockout, rate limits.
- Authorization: admin / operator / viewer roles enforced in the backend; out-of-scope resources return 404.
- Networks: private-range CIDR validation (/16–/29), overlap rejection including concurrent creates, address allocation guarded by a unique database index.
- Enrollment: 15-minute single-use tokens stored as hashes; Ed25519 proof of possession; duplicate identity and replay rejection; revocation.
- Telemetry schema v1: interface state and source, WireGuard peers, endpoint candidates, measurements where unavailable means null (never 0).
- Health model, topology, incident detector, read-only diagnostic tools, deterministic planner, policy engine, approvals, command dispatch, post-execution verification, audit, Socket.IO live updates.

4.3 Networking agent (Go)

Command	What it does
enroll	Generates Ed25519 + WireGuard keys locally and enrolls with a one-time token (read from a prompt, never a flag)
run	Heartbeats, telemetry, TCP probes, approved commands; read-only unless privileged mode is enabled
status / doctor	Local inspection; read-only preflight that explains exactly what is missing for a real tunnel
export-config	Writes a tunnel file for the WireGuard app or wg-quick (only /32 peer routes, no DNS, no default route)
lab	Runs a real WireGuard tunnel (wireguard-go) between two enrolled agents on one host, against the live control plane
probe / apply	Bounded TCP probe; explicit privileged apply with read-back verification

4.4 Diagnostic service (Python)

- Strict request/response schemas with no confidence field.

- Providers: mock (default, output labelled “mock”) and Groq (JSON mode). Only evidence summaries are sent to Groq, never IP addresses or endpoints.
- Grounding validator: rejects cited evidence that was not collected, non-allowlisted actions and confidence scores.

5. Status matrix

Legend: Done & tested · Partially verified · Ready, manual run pending · Not started

Area	Status	Evidence
Login, sessions, CSRF, lockout, RBAC, scoped access	**Done & tested**	auth.test.js
Sign up and register-this-PC onboarding	**Done & tested**	signup.test.js + browser check
Networks: CIDR, overlap, unique addresses	**Done & tested**	networks.test.js, ipv4.test.js
Enrollment, proof of possession, revocation	**Done & tested**	enrollment.test.js
Signed agent protocol (Go ↔ Node)	**Done & tested**	Shared signature test vector + integration test
Heartbeats, telemetry v1, honest health model	**Done & tested**	telemetry.test.js
Topology: observed vs configured	**Done & tested**	telemetry.test.js + browser check
Incidents → diagnosis → policy → approval → verify	**Done & tested**	incidents.test.js
Audit log	**Done & tested**	incidents.test.js, auth.test.js
Web console (all pages)	**Done & tested**	Production build + browser walkthrough
Go agent: build, unit tests, live integration	**Done & tested**	20 Go tests + integration test
Real WireGuard tunnel, two agents (lab)	**Done & tested**	nexlink-agent lab runs; approved test 0.23 ms / 0% loss
export-config, doctor, LAN endpoint ranking	**Partially verified**	Two agents on one PC; not yet on two PCs
Groq LLM diagnosis	**Partially verified**	15 tests with mocked Groq; live call needs key
Kernel / WireGuardNT interface (WireGuard app)	**Ready, manual run pending**	Needs WireGuard app + Administrator
Tunnel between two physical PCs	**Ready, manual run pending**	Runbook docs/two-pc-test.md
Relay fallback and NAT traversal	**Not started**	Designed in docs/network-protocols.md
TLS deployment, MFA, OS keychain keys, CI	**Not started**	Roadmap

6. Verification results

6.1 Automated tests

Suite	Command	Result	Category
Control plane	cd control-plane && npm test	**59 / 59**	Integration vs ephemeral MongoDB; simulated agent with real keys
Go agent	cd networking-agent && go test ./...	**20 / 20**	Unit + real wireguard-go tunnel on loopback
Agent ↔ control plane	go test -tags integration ./tests/	**Passed**	Real agent vs running control plane
Diagnostic service	python -m pytest -q	**15 / 15**	Contracts; Groq mocked
Console build	npx next build	**Passed**	12 routes
Two-PC WireGuard	docs/two-pc-test.md	**Pending**	Separate physical machines

6.2 Real WireGuard tunnel (lab)

Setup: two agents (pc-a 10.100.0.4 and pc-b 10.100.0.5), each enrolled with its own keys into the same network, ran on one Windows PC. Each used a wireguard-go device — the official userspace WireGuard implementation — with a private network stack, connected over loopback UDP. Desired peers, telemetry and actions all went through the live control plane exactly as in production. No Windows adapter, route or DNS setting was changed.

Check	Observed result
WireGuard handshake	Completed on both sides; refreshed every ~2 minutes
Probe RTT through the tunnel (75 s run)	0.19 – 0.63 ms, every 10 s, both directions
8-minute run	No errors; 94 RTT samples from <0.01 ms to 5.9 ms (peak while the control plane restarted); handshakes refreshed throughout
Console topology	pc-a ⇄ pc-b “connected · direct”; links to offline devices fell back to “configured”
Device health	“degraded” — correct, because each had only 1 of 3 configured peers connected
Approved run_connectivity_test (pc-a → pc-b)	Executed by the agent through the tunnel: RTT 0.23 ms, jitter 0.31 ms, loss 0%
Verification outcome	Successful — 3/3 measurements valid; test incident stays open (drills never auto-resolve)

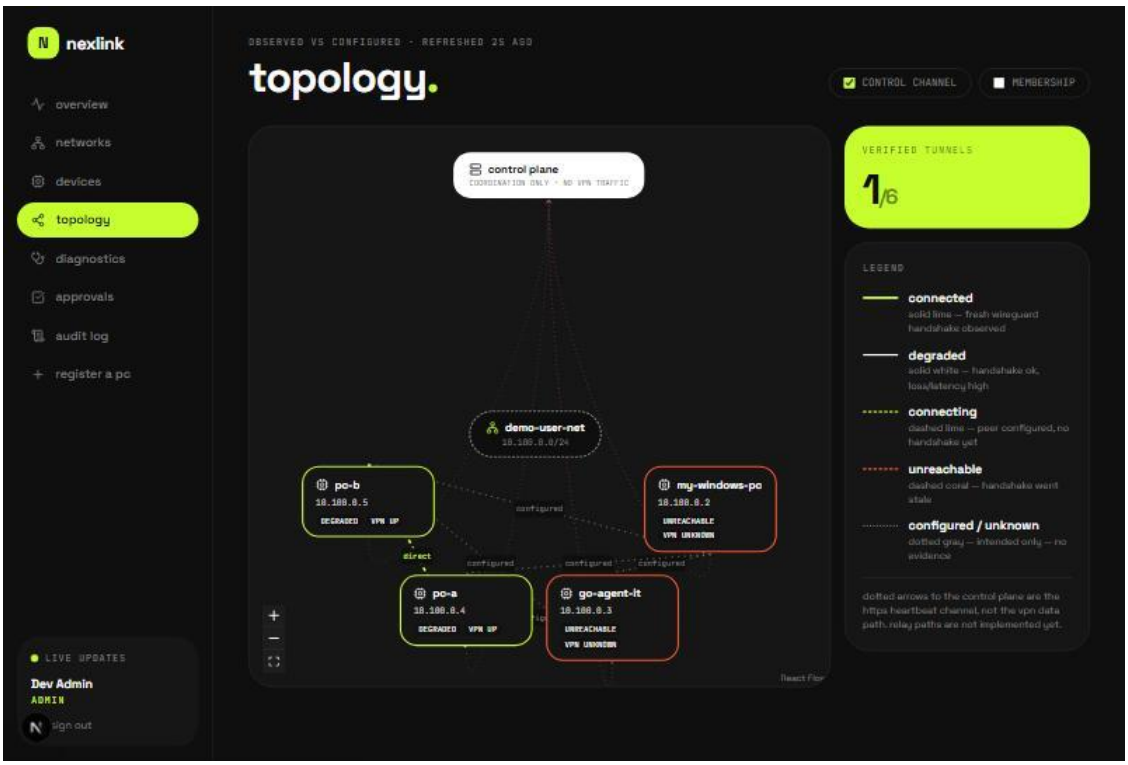


Figure 6 — Topology during the lab run: the pc-a ⇄ pc-b edge is verified by a handshake.

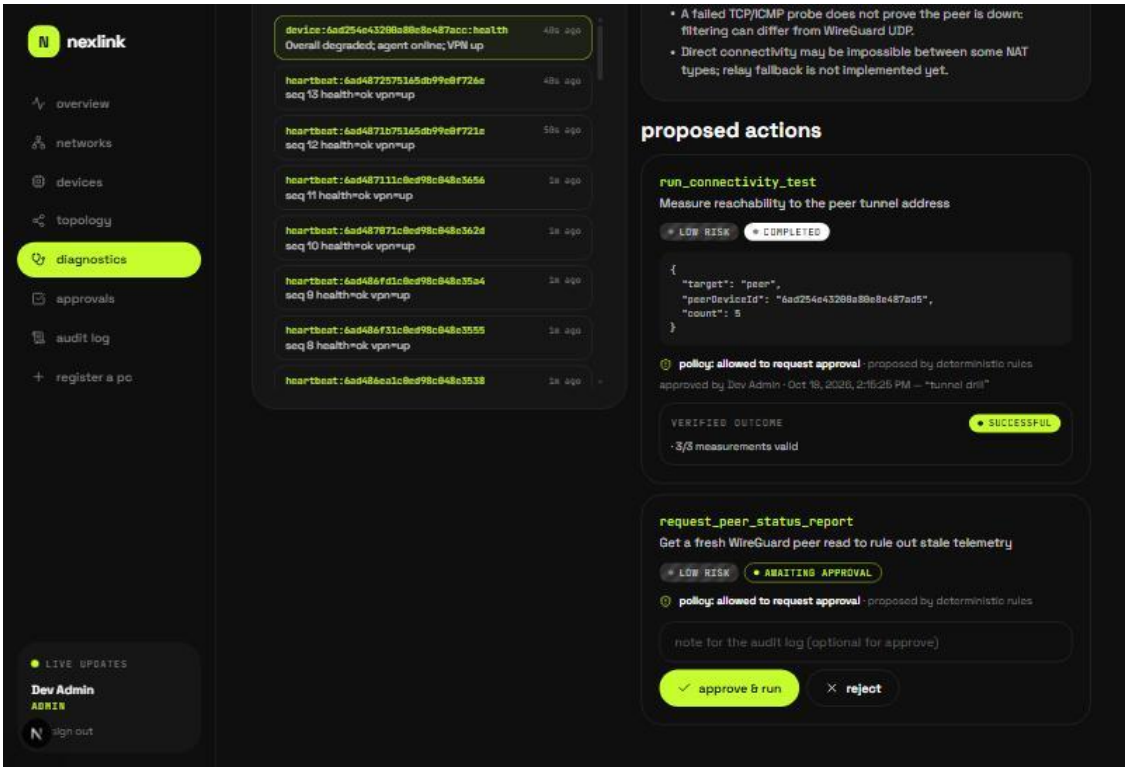


Figure 7 — Approved connectivity test executed through the tunnel and verified as successful.

Scope of the lab result

It proves the full path enrollment → desired config → handshake → telemetry → policy-gated action → verification with the real WireGuard protocol. It does not yet prove connectivity across two physical machines, a real LAN, or NAT; that is the pending two-PC test.

6.3 Browser checks

- Sign up, register this PC, live flip to “registered” when the agent enrolled.
- Network creation with CIDR errors shown inline; enrollment token issued once and not stored in the browser.
- Device page showing unknown (not healthy) when WireGuard could not be read.
- Diagnostics, approval, and an honest “timed out” verification when the agent did not execute a command.
- “Ask AI” round trip through the diagnostic service (mock provider), output labelled as mock.

6.4 Bugs found and fixed during verification

Bug	Fix
Agent CLI flags (--server, --state-dir) were silently ignored	Flags now parse into the shared config (pointer); caught by the two-agent test
Agent advertised a Radmin VPN address instead of the Wi-Fi LAN address	Endpoint candidates ranked: private LAN first, known VPN/virtual adapters last
Links to peers removed from an interface still showed “connecting”	Stale peer observations are dropped when a real WireGuard report no longer lists the peer
README rendered as raw text on GitHub	Removed a UTF-16 line (NUL bytes) appended by a PowerShell echo
Live-update socket died if the page loaded before login	Socket connects only after a session exists and retries after auth errors
DELETE requests rejected as “not JSON”	JSON-only check applies only when a body is present
Concurrent enrollments could exhaust allocation retries	Retries spread over the lowest free addresses
Approved peer tests could not reach a userspace tunnel	Executor accepts a dial hook used by lab mode
Status badge unreadable on white cards; page overflow on long refs	Solid badge tone; grid min-width fixes
Overlapping label in the architecture diagram	Label removed
Test command did not find test files on Node 22	Glob pattern in npm test

7. Security model

- Sessions in HttpOnly, SameSite=Strict cookies; CSRF token kept in memory only; scrypt hashing; lockout and rate limits.
- Every agent request is Ed25519-signed over method, path, timestamp, nonce, device id and body hash; nonces are single-use.
- Private keys are generated on the device and never stored in the database; enrollment tokens are stored only as hashes.
- RBAC and network scope enforced in the backend; strict schemas reject unknown fields (mass-assignment protection).
- The agent applies only /32 peer routes inside the network range and IP-literal endpoints; privileged mode is a local opt-in.
- AI output is treated as untrusted input: schema, evidence grounding, action allowlist, policy engine, human approval, agent-side allowlist.
- Audit log is append-only from the API's point of view; secret-like metadata is redacted.
- Repository hygiene: .env, dev credentials, keys, exported tunnel files, build outputs and the bundled Go toolchain are git-ignored.

8. What is not in the project yet

Item	Status	What is needed / next step
Tunnel between two physical PCs	**Ready, manual run pending**	Second PC, WireGuard app, firewall rules — follow docs/two-pc-test.md
Kernel / WireGuardNT interface	**Ready, manual run pending**	Install WireGuard for Windows; import export-config output; run agent as Administrator
Live Groq diagnosis	**Ready, manual run pending**	Set GROQ_API_KEY and NEXLINK_DIAG_PROVIDER=groq; review output quality
Relay fallback / NAT traversal	**Not started**	UDP relay, request_relay_fallback action, path=relay reporting
Application-level checks (layer 5)	**Not started**	Service probes defined per network
TLS deployment guide / reverse proxy	**Not started**	Required before any shared deployment
MFA, SSO, password reset	**Not started**	Account hardening
Agent keys in OS keychain / TPM	**Not started**	Today keys are files protected by file permissions
CI pipeline (GitHub Actions)	**Not started**	Run the three test suites on every push
Multi-instance scaling	**Not started**	Shared rate-limit store; Socket.IO adapter

Known limitations and environment notes

- Socket.IO rooms are computed at connect time; a user added to a network later needs a page reload for live updates (data still refreshes on a timer).
- Docker Desktop on the development PC fails to start because of a stale socket file (%LOCALAPPDATA%\Docker\run\dockerInference) that needs an Administrator to delete. The project does not depend on Docker.
- Plain HTTP between agent and control plane is allowed on a lab LAN only with NEXLINK_ALLOW_HTTP=true; requests are signed but not encrypted.
- The test network used for verification contains two offline test devices, which is why lab devices show “degraded” (1 of 3 peers connected).

9. How to run

Control plane and console

```
cd control-plane && cp .env.example .env && npm install
npm run dev:mongo        # dev-only MongoDB on 127.0.0.1:27017
npm run dev              # http://localhost:4000
cd ../frontend && cp .env.example .env.local && npm install && npm run dev
```

Open http://localhost:3000, sign up, then “register a pc”.

Agent (Administrator PowerShell)

```
cd networking-agent\bin
.\nexlink-agent.exe enroll --server http://localhost:4000
.\nexlink-agent.exe run
```

Real tunnel on one PC

```
.\nexlink-agent.exe lab --server http://localhost:4000 --a .\lab-a --b .\lab-b --duration 5m
```

Tests

```
cd control-plane && npm test
cd networking-agent && go test ./...
cd diagnostic-service && python -m pytest -q
```

Roadmap

1. Run the two-PC tunnel test with the WireGuard app (docs/two-pc-test.md).
2. Run a live Groq diagnosis and review its quality.
3. Implement relay fallback and NAT traversal.
4. Add CI, a TLS deployment guide, MFA and OS keychain key storage.

Appendix A — Repository layout

Path	Contents
frontend/	Next.js console (JavaScript, Tailwind, React Flow, Socket.IO)
control-plane/	Express + MongoDB: auth, networks, enrollment, telemetry, incidents, policy, audit; tests/
networking-agent/	Go agent: cmd/nexlink-agent and internal/* packages; tests/ (integration)
diagnostic-service/	FastAPI: schemas, providers (mock, Groq), validators, planners; tests/
docs/	architecture, threat model, network protocols, API contracts, database schema, roadmap, two-PC runbook, images
docker-compose.yml	MongoDB, optional control plane and diagnostic service

Appendix B — Documentation index

Document	What is inside
docs/architecture.md	Components, health model, diagnostic loop, MongoDB constraints
docs/network-protocols.md	Enrollment, request signing, telemetry v1, probes, NAT/relay design
docs/api-contracts.md	Every REST endpoint with its permission
docs/database-schema.md	Collections, indexes, retention
docs/threat-model.md	Threats, mitigations, known gaps
docs/two-pc-test.md	Runbook for the two-PC tunnel, including a one-PC rehearsal
docs/implementation-roadmap.md	Stage status and test categories